

1.(b). Data and a pointer to the next node.

2(c). NULL

3.(b). Self-referential structure

4(b). $\text{head} == \text{NULL}$

5.(b). $\text{newnode} \rightarrow \text{next} = \text{head}; \text{head} = \text{newnode};$

6.(b). $\text{newnode} \rightarrow \text{next} = \text{head}; \text{head} = \text{newnode};$

7.(c). Traversing from tail to head

8(b). n

9(c). Copying the data of $p \rightarrow \text{next}$ into p, then deleting $p \rightarrow \text{next}$

10(b). While($p! = \text{NULL}$)

Name : Atif Intezar

Roll : UG/04/BTCSE/2025/041

Section : A

11(b). $p \rightarrow \text{next} == \text{NULL}$

~~12(e). Floyd's cycle detection algorithm~~

12(c) Two pointers advancing at different rates

13(c) Three

14(b). Exactly one node

15(b). $p(\text{struct node}^*) \text{malloc}(\text{sizeof}(\text{struct node}));$

16(b). Avoid special-case handling for insertion and deletion at the head.

17(c). The first node

18(b). It does not support random (indexed) access.

19.(b). Loss of access to all nodes, causing a memory leak.

20(a). One pointer and the node count.

21(c). The Last node

22(b). Nodes may be scattered anywhere in memory.

23. struct node {

int data;

struct node* next;

};

Name : Atif Intezar

Roll : UG/04/BTCSE/2025/041

Section : A

24. 30 50 70

28.(i) node newNode = new node(⁷~~data~~);
newNode.next = head;
head = newNode;

(iii) if (head == null) {
 head = newNode;
 return;
}
Node temp = head;
While (temp.next != null) {
 temp = temp.next;
}
temp.next = newNode;

Name : Atif Intezar
Roll : UG/04/BTCSE/2025/041
Section : A

5 6 8 9

Name : Atif Intezar
Roll : UG/04/BTCSE/2025/041
Section : A